Sampling and Logits Processing

Deep Dive into SGLang — Chapter 14

Yin Yang

Foundation Books Project

Where this chapter sits

This is the chapter that turns "a tensor of scores" into "the token this request asked for."

- The model forward pass produces hidden states; the runtime owes the user tokens, logprobs, and streamable output.
- Sampling and logits processing are the bridge — a row-local policy applied inside a batched tensor.
- Two rows can share one decode batch: one wants top-p, the other wants greedy output with logprob records.

Goal: by the end you can follow one logits row from hidden state to token id, and say which request owns every transform along the way.

Roadmap

The token-choice problem

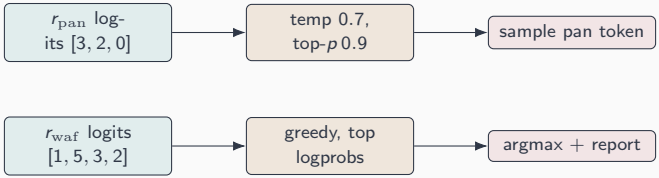
The row transform

From concept to code

Correctness and boundary

The token-choice problem

Two policies, one batch



The systems problem is row alignment: the sampler must never apply r_{pan}'s temperature to r_{waf}'s row, or attach r_{waf}'s logprob record to r_{pan}'s token.

Four nearby names split the work

Logits processor turns selected hidden rows into vocabulary logits and optional logprob records — $L_t = P_{\text{logits}}(H_t, M_t)$.

Logits metadata the per-step contract: which rows to score, which records to return.

Sampling policy the request-owned rule σ_r: one logits row $\rightarrow$ one token decision + logprobs.

Sampling batch state tensorizes per-row policy (temps, filters, masks, penalties) in *visible batch order*.

Keep these four straight and every later slide is just bookkeeping on one row.

The row transform

The per-row pipeline, as algebra

For request row i, raw logits ℓ_i become a distribution π_i:

$$a_i = \Phi_i(\ell_i; \Delta_i), \quad \mathbf{p}_i = \mathrm{softmax}(a_i/\tau_i), \quad \pi_i = \mathrm{Normalize}\big(\mathrm{Filter}_{k_i, p_i^{\mathrm{top}}, m_i}(\mathbf{p}_i)\big).$$

- Φ_i: penalties, mask values, logit bias, custom processors (its additive part is Δ_i; repetition scaling is nonlinear).
- τ_i: temperature — $\tau_i < 1$ sharpens, $\tau_i > 1$ flattens.
- Filter: top-k_i, top-p_i^{top}, min-p_i.

Every parameter carries the same row index i as the logits. That is the whole invariant, written once.

Worked example: temperature then nucleus

Request A: logits $[3, 2, 0]$, temperature $\tau = 0.7$, top-$p = 0.9$.

$$a/\tau = [4.29,\ 2.86,\ 0], \qquad \mathbf{p} = \mathrm{softmax}(a/\tau) \approx [0.797,\ 0.191,\ 0.011].$$

Sort by probability (already sorted), take the shortest prefix reaching 0.9:

$$0.797 < 0.9, \quad 0.797 + 0.191 = 0.988 \geq 0.9 \ \Rightarrow\ \text{keep } \{0, 1\}.$$

Renormalize over the kept set:

$$\pi \approx [0.807,\ 0.193].$$

Greedy row $B = [1, 5, 3, 2]$ just takes index 1. Same tensor, two policies — applied to the right rows.

Penalties are stateful, request-local tensors

Let $c_r(v)$ be token v's count in request r's output so far. SGLang applies additive penalties first, then sign-dependent repetition scaling:

$$\Delta_v = -\big(\underbrace{\alpha_r c_r(v)}_{\text{frequency}} + \underbrace{\beta_r \mathbf{1}[c_r(v) > 0]}_{\text{presence}}\big), \qquad \ell_v \leftarrow \begin{cases} \ell_v / \rho_r & \ell_v > 0 \\ \ell_v \, \rho_r & \ell_v < 0. \end{cases}$$

Concrete row ($\alpha = 0.3, \beta = 0.8, \rho = 1.2$): token 42 seen twice, $\ell_{42} = 2.4$:

$$\Delta_{42} = -(0.3 \cdot 2 + 0.8) = -1.4, \quad 1.0/1.2 = 0.83.$$

This is why penalties live as per-row accumulators, not scalar knobs.

From concept to code

From concept to code: the sampler's spine

The whole row transform has a concrete home in `Sampler.forward`.

from `python/sglang/srt/layers/sampler.py`

```
logits = logits_output.next_token_logits
logits = self._preprocess_logits(logits, sampling_info)  # custom procs, NaN
if sampling_info.is_all_greedy:
    batch_next_token_ids = torch.argmax(logits, -1)       # greedy fast path
else:
    logits.div_(sampling_info.temperatures)               # per-row temperature
    logits[:] = torch.softmax(logits, dim=-1)
    batch_next_token_ids = self._sample_from_probs(...)    # top-k/top-p/min-p
```

Row *i* of `logits` meets row *i* of `sampling_info`: argmax and sampling run over the *vocabulary* axis, never the batch axis.

Concept-to-code map for the whole chapter

- **Logits metadata (row & record selection)** → `LogitsMetadata in layers/logits_processor.py`
- **Per-request sampling policy tensors** → `SamplingBatchInfo in sampling/sampling_batch_info.py`
- **Validated request knobs** → `SamplingParams in sampling/sampling_params.py`
- **Token choice** σ_r → `Sampler in layers/sampler.py`
- **Stateful penalties** → `penaltylib/ frequency, presence, repetition`
- **Visible token + logprob record** → `LogitsProcessorOutput fields`

Each object preserves the same visible row order — that is the spine of the chapter.

Correctness and boundary

The one invariant the chapter defends

Invariant (Sampling-row alignment)
A sampling step is valid only when request identity, logits row, sampling parameters, masks, penalties, logprob requests, TP reconstruction, custom processors, and batch order all describe the same visible request row.

The failure mode is quiet: a misaligned row still returns a legal token id. The symptom is a mismatched *side channel* — wrong logprob, wrong grammar, wrong penalty history — not a crash.

What sampling owns — and what it does not

Sampling owns

- scored rows → token ids;
- temperature, top-k/p/min-p;
- penalties, logit bias;
- logprob record extraction;
- TP token-id agreement.

Adjacent owners

- model: hidden states, raw logits;
- structured output: grammar masks;
- parser: interprets emitted text;
- protocol: formats logprobs;
- collectives: rank mechanics.

The rule is one-way: grammar can make a token impossible before sampling, TP can force agreement after — but neither *chooses* the row's token.

What to carry forward

1. Token choice is a **row-aligned transformation**; each knob has an owner, valid only while attached to its request row.
2. The math is on the slide: **penalties** $\rightarrow$ **temperature** $\rightarrow$ **filters** $\rightarrow$ **draw**, every parameter row-indexed.
3. **Penalties are stateful**: the next logits row depends on tokens already committed to the same request.
4. **Logprobs** are scores derived from the same row — not a separate policy.
5. Every concept has a **home in the SGLang source**; row order is preserved end to end.

Next: Section 1 makes the four objects precise, then Section 2 walks one row from hidden state to committed token.